



Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Para la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso como, por ejemplo, los mapas de Karnaugh o los sistemas de representación. Por lo tanto, se recomienda que les deis un vistazo.

Competencias

Conocer la organización general de un computador como circuito digital y conocer las características distintivas de la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la selección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

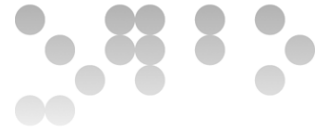
Recursos

Los recursos que se recomienda usar para esta práctica son los siguientes:

- **Básicos:** El módulo 5 de los materiales.
- **Complementarios:** VerilCIRC, VerilCHART y la Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no serán puntuadas.
- La valoración está indicada en cada uno de los subapartados.



Formado y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas entregadas junto con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **16 de diciembre**, a las 24 horas.

Solución de la práctica propuesta

PRIMERA PARTE [65 %]

Dada la máquina de estados siguiente:

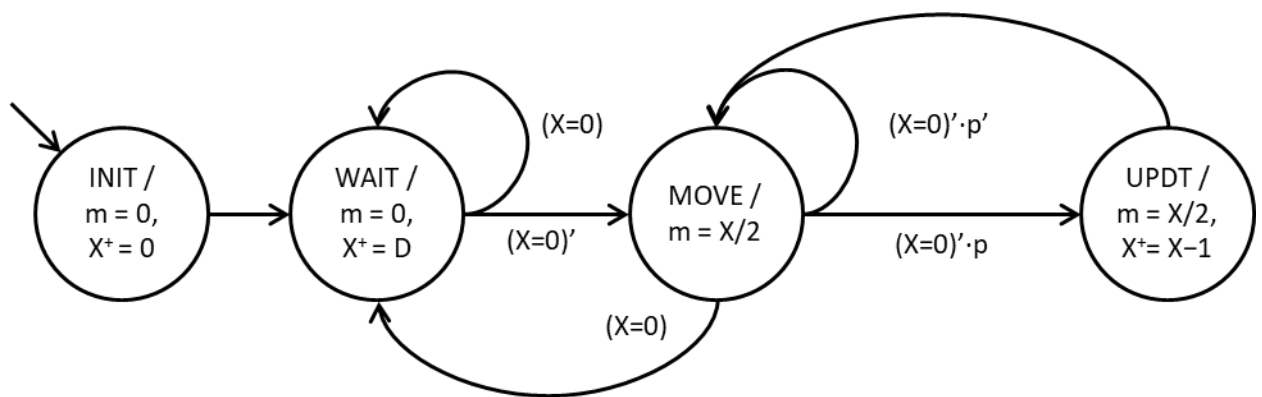
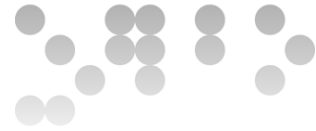


Fig. 1. EFSM de un controlador de movimiento adelante de un robot.

Teniendo presente que la operación $X/2$ es una división entera, se pide que:

- a) [15%] A partir del grafo de la EFSM de la fig. 1 completad, en el cronograma siguiente, la evolución de X , m y del estado (E) en que se encuentra la máquina en cada ciclo de reloj.

Nota: Tenéis el ejercicio disponible en VerilChart, donde la señal $(X=0)$ es $X0$; los valores de X (que tiene 4 bits) se representan en hexadecimal, y el estado por E .

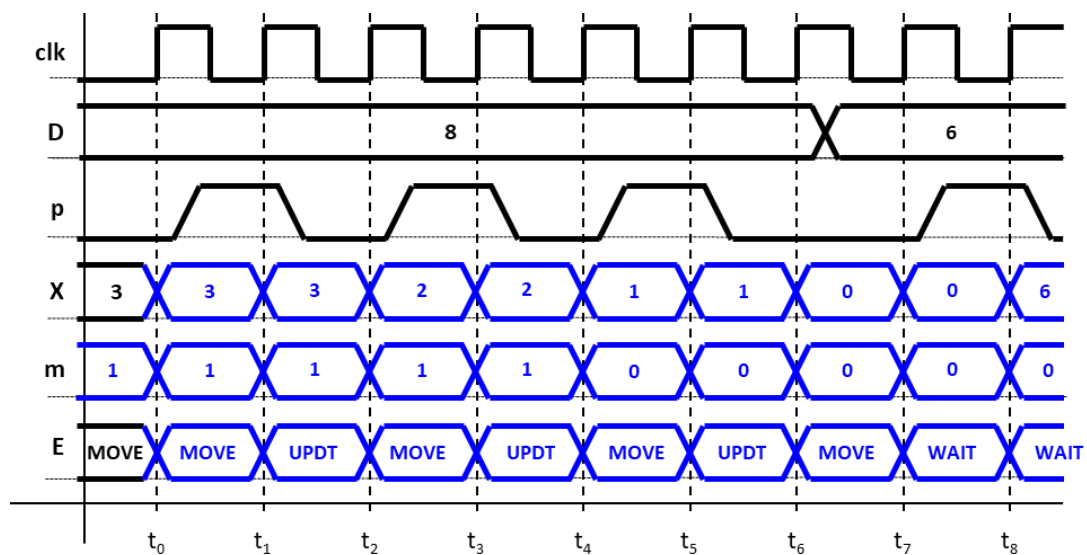


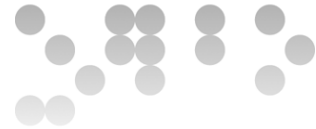
A partir de las condiciones iniciales que hay en el cronograma ($E=MOVE$ y $X=3$) se puede determinar el estado siguiente y el nuevo valor de X . Al ser una señal de salida, el valor de m depende de forma exclusiva y combinacional del estado E y del valor de X en este estado.

X es una variable y, por lo tanto, el valor que se calcula en el estado actual será efectivo en el ciclo de reloj siguiente, es decir, en cada estado se calcula el valor que tendrá en el estado siguiente.

Así pues, para determinar el estado y el valor de X a partir de t_0 y hasta t_1 , debemos fijarnos que, estando en $MOVE$, el valor de X para el estado siguiente se mantiene, es decir, vuelve a ser 3. Y el estado siguiente vuelve a ser $MOVE$, porque en t_0 la señal p es 0 y la señal ($X=0$) también es 0, dado que X es 3.

Siguiendo este procedimiento, se pueden determinar los valores para el periodo t_1-t_2 a partir de los del periodo anterior y se puede ir repitiendo el procedimiento hasta completar el cronograma, tal como se muestra en la figura siguiente.





- b) [15%] Obtened las tablas de transiciones y de salidas de la EFSM de la fig. 1, así como las codificaciones binarias de estados y salidas correspondientes.

En el grafo hay dos señales de entrada, p y $(X=0)$. Además, tiene la variable X y la señal de salida m , para las que hay que determinar los valores que toman.

Las tablas de transiciones y salidas que se obtienen a partir del grafo son:

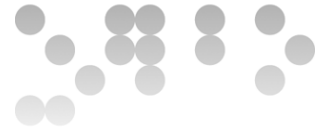
Estado actual	Entrada	Estado siguiente	Estado	Salidas
INIT	x	WAIT	INIT	$X^+ = 0, m = 0$
WAIT	$(X=0)'$	MOVE	WAIT	$X^+ = D, m = 0$
WAIT	$(X=0)$	WAIT	MOVE	$X^+ = X, m = X/2$
MOVE	$(X=0)' \cdot p'$	MOVE	UPDT	$X^+ = X-1, m = X/2$
MOVE	$(X=0)' \cdot p$	UPDT		
MOVE	$(X=0)$	WAIT		
UPDT	x	MOVE		
GC	$(N=1)$	WC		

Para la codificación binaria, hay que asignar a cada estado un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde el 0 para el estado inicial hasta el número de estados que haya menos 1: 00 para INIT, 01 para WAIT, 10 para MOVE y 11 para UPDT.

Para elegir el valor siguiente que se asigna a la variable X hace falta un selector de dos bits para poder elegir entre cuatro valores posibles: 0, D , X , o $X-1$. Como cada valor se elige en un estado diferente, se puede usar el código del estado como selector:

Operación	Estado	Efecto sobre la variable
$X^+ = 0$	00	Ponerlo a 0
$X^+ = D$	01	Asignarle el valor de la entrada D
$X^+ = X$	10	Mantener el valor
$X^+ = X + 1$	11	Sumarle 1

El valor de la salida m se determina combinatorialmente a partir del estado y de X . El valor que tiene se puede seleccionar de manera similar a como se determina el valor para el estado siguiente de X : con el bit de más peso del código del estado.



Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondientes son:

Estado actual		Entrada		Estado ⁺
Símbolo	q_1q_0	$(X=0)$	p	$q_1^+q_0^+$
INIT	00	x	x	01
WAIT	01	0	x	10
WAIT	01	1	x	01
MOVE	10	0	0	10
MOVE	10	0	1	11
MOVE	10	1	x	01
UPDT	11	x	x	10

Estado		Selector	
Símbolo	q_1q_0	q_1q_0	q_1
INIT	00	00	0
WAIT	01	01	0
MOVE	10	10	1
UPDT	11	11	1

Fijaos que, en la columna “Selector” se ha repetido el contenido del estado. Para seleccionar el nuevo valor de X se usa el código del estado y para seleccionar el valor de la salida m se usa q_1 , el bit de más peso del código del estado. Si se usan señales diferentes a estas, también es correcto. Tened presente, además, que, según la codificación elegida para los estados y para el selector, el resultado de la tabla anterior puede variar.

El modelo de un controlador de movimiento de un robot para hacer que vaya a una posición indicada por un ángulo A y una distancia D se representa con la EFSM siguiente.

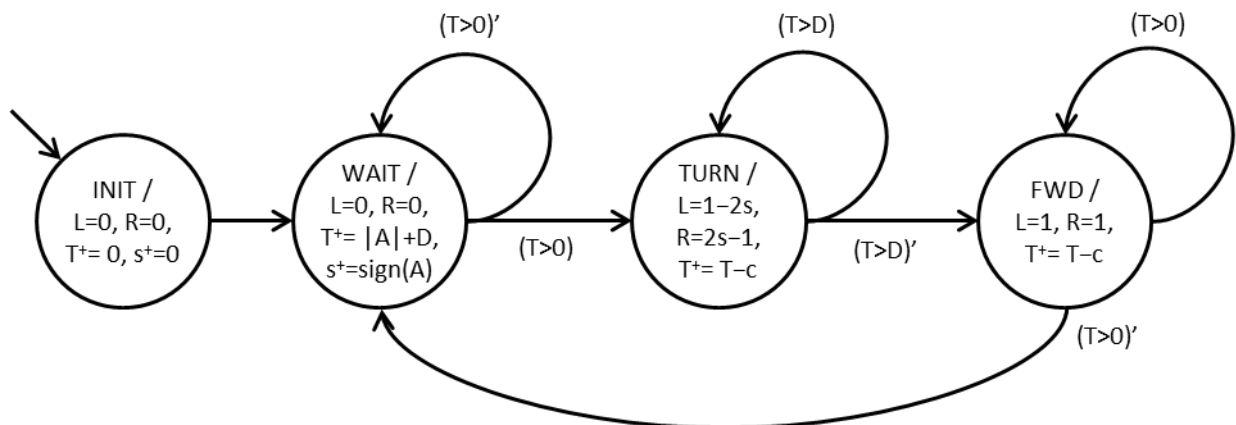
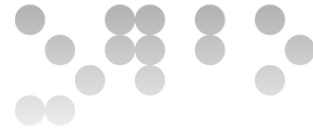


Fig. 2. EFSM de un controlador de movimiento de un robot.



Tanto la entrada A como la D son de 4 bits, con A representando valores enteros en signo y magnitud y D , valores naturales. La operación $|A|$ calcula el valor absoluto de A y $\text{sign}(A)$ devuelve el bit que indica el signo de A .

Las variables T y s almacenan el total de unidades a girar o avanzar y el bit de signo de A , respectivamente.

Hay una entrada de un bit c , que se pone a 1 cuando las ruedas han recorrido 1 cm de distancia lineal, que se resta a T .

Las señales de salida L y R indican a los motores de las ruedas si tienen que girar adelante (1), atrás (−1) o estar parados (0). Son valores que se representan en complemento a 2. Las operaciones $1-2s$ y $2s-1$ dan 1 y −1 o −1 y 1, según si s es 0 o 1, respectivamente.

Teniendo en cuenta lo que se ha indicado previamente, se pide que:

- c) [15 %] A partir de las tablas de transiciones y de salidas del grafo de la fig. 2 que se dan a continuación, implementad la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creáis convenientes. Especificad y justificad las dimensiones de la/las ROM que uséis e indicad el contenido en binario, indicando a que corresponde cada bit, y en hexadecimal.

Estado actual	Entrada		Estado ⁺
	($T>0$)	($T>D$)	
INIT	x	x	WAIT
WAIT	0	x	WAIT
WAIT	1	x	TURN
TURN	x	0	FWD
TURN	x	1	TURN
FWD	0	x	WAIT
FWD	1	x	FWD

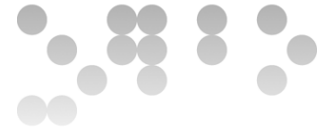
Estado		Salidas	
Símbolo	q_1q_0	s_LR	s_sT
INIT	00	00	00
WAIT	01	00	01
TURN	10	10	10
FWD	11	11	10

Las señales s_LR y s_sT se usan como selectores para elegir qué resultados de los cálculos que se hacen se asignan a L y a R , y a s y a T , respectivamente.

Nota: Tenéis el ejercicio disponible en VerilCIRC y, si queréis comprobar previamente la/las ROM, la tenéis disponible en VerilChart. En este entorno, el estado es E , ($T>0$) es $T0$ y ($T>D$) es TD .

Para implementar la unidad de control con una ROM hace falta, primero, determinar el tamaño y, después, calcular el contenido a partir de la tabla de transiciones correspondiente.

El bus de direcciones de la ROM debe ser de 4 bits, puesto que la dirección se forma con los bits que codifican el estado (son 2, en este caso) y los de las señales

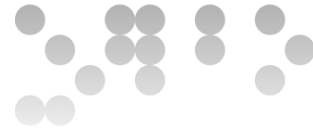


de entrada (2 más). Por lo tanto, la ROM tiene que disponer de un total de $2^4 = 16$ posiciones de memoria.

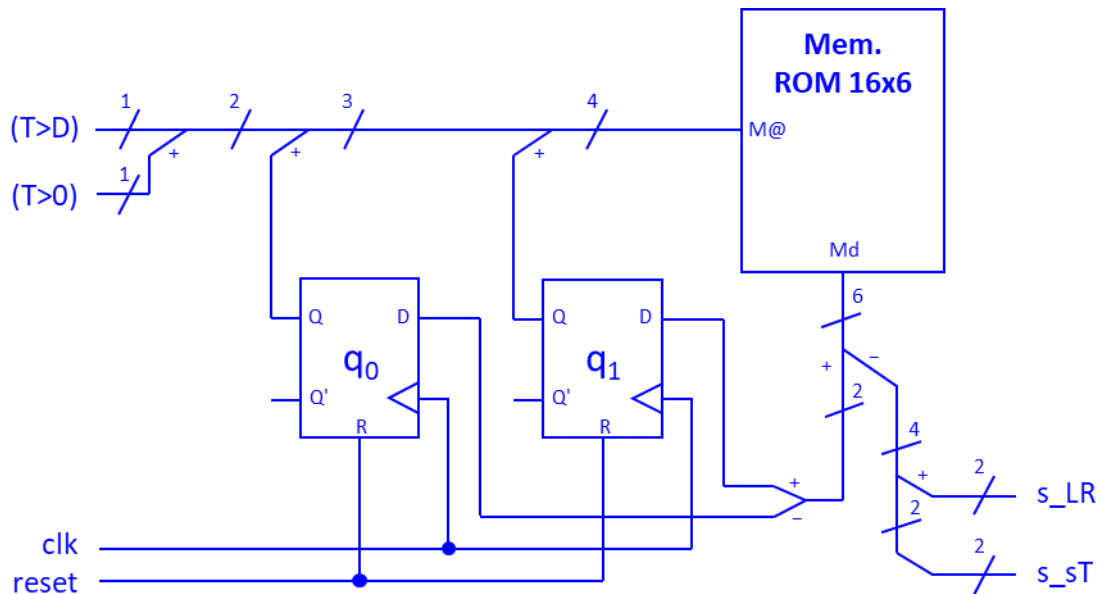
En cada posición se almacena el código del estado siguiente y el valor, en el estado actual, de cada una de las señales de salida. Por lo tanto, harán falta 2 bits para el estado y 4 bits para las salidas (s_{LR} y s_{sT}), cosa que hace un total de 6 bits. Así pues, la ROM debe ser de 16 posiciones de 6 bits.

El contenido de la ROM se determina partir de las tablas de transiciones y salidas, combinando los códigos binarios correspondientes. Así pues, la ROM debe tener el contenido siguiente:

Posición de memoria			Salidas			Codificación hexadecimal
q_1q_0	(T>0)	(T>D)	$q_1^+q_0^+$	s_{LR}	s_{sT}	
00	0	0	01	00	00	10h
00	0	1	01	00	00	10h
00	1	0	01	00	00	10h
00	1	1	01	00	00	10h
01	0	0	01	00	01	11h
01	0	1	01	00	01	11h
01	1	0	10	00	01	21h
01	1	1	10	00	01	21h
10	0	0	11	10	10	3Ah
10	0	1	10	10	10	2Ah
10	1	0	11	10	10	3Ah
10	1	1	10	10	10	2Ah
11	0	0	01	11	10	1Eh
11	0	1	01	11	10	1Eh
11	1	0	11	11	10	3Eh
11	1	1	11	11	10	3Eh

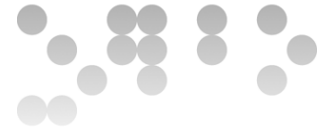


La figura siguiente muestra la implementación con una ROM de estas dimensiones. La entrada $M@$ de la ROM se configura con el valor del estado actual (los 2 bits de más peso) y el valor de las entradas (los 2 bits de menos peso). Para guardar el estado actual se usan 2 biestables.



La otra opción es usando dos ROM, una para el cálculo del estado siguiente y la otra para las salidas. En este caso, la ROM de cálculo del estado siguiente tiene el mismo número de bits de dirección que el anterior (es decir, 4) pero solo dos bits para el contenido, puesto que solo se almacena el código del estado siguiente:

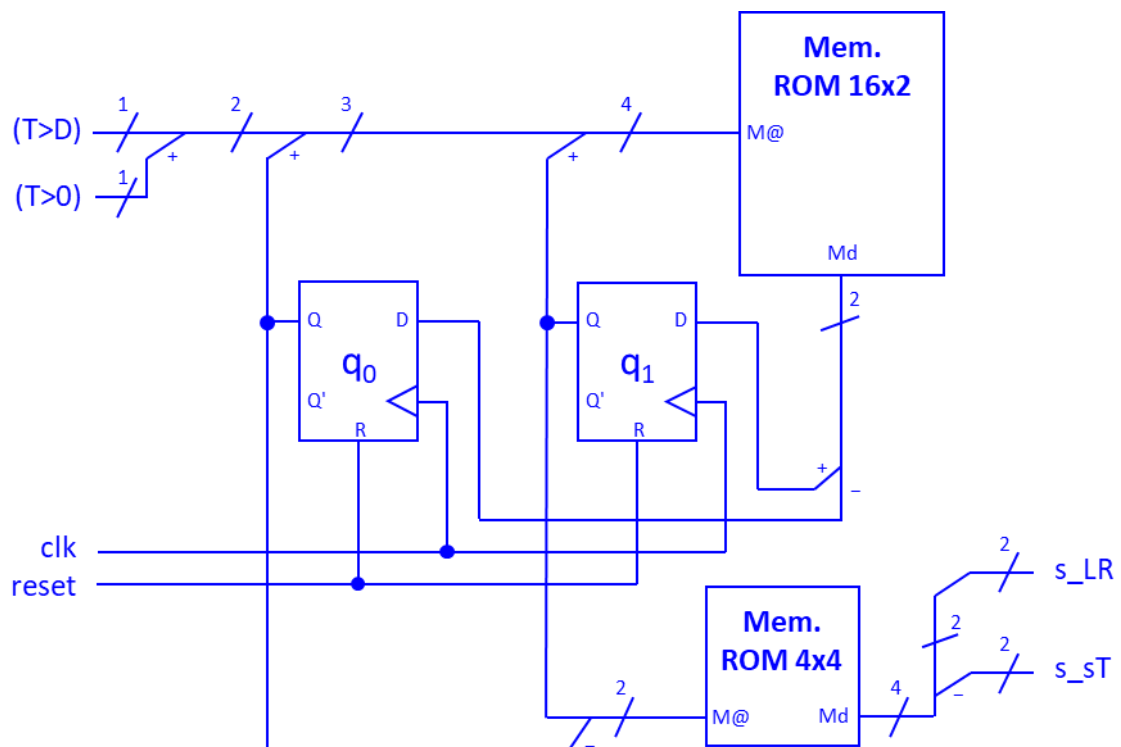
Posición de memoria			Salidas	Codificación hexadecimal
q_1q_0	($T>0$)	($T>D$)	$q_1^+q_0^+$	
00	0	0	01	1h
00	0	1	01	1h
00	1	0	01	1h
00	1	1	01	1h
01	0	0	01	1h
01	0	1	01	1h
01	1	0	10	2h
01	1	1	10	2h
10	0	0	11	3h
10	0	1	10	2h
10	1	0	11	3h
10	1	1	10	2h
11	0	0	01	1h
11	0	1	01	1h
11	1	0	11	3h
11	1	1	11	3h



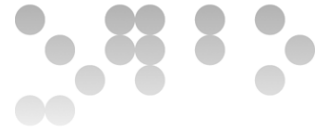
La ROM del cálculo de las salidas tiene direcciones de dos bits (el código del estado) y contenidos de 4 bits para las salidas:

Posición de memoria	Salidas		Codificación hexadecimal
	q_1q_0	s_LR s_sT	
00	00	00	0h
01	00	01	1h
10	10	10	Ah
11	11	10	Eh

El circuito que usa dos ROM es el siguiente:



Con esta última versión se usan menos bits de memoria ROM: de 96 (16x6) con una única ROM a 48 (16x2 + 4x4).



- d) [20%] Implementad el circuito completo de la EFSM de la fig. 2: Construid el camino de datos usando las puertas lógicas y los bloques necesarios e incorporad la unidad de control obtenida en el apartado anterior. Indicad y razonad, para cada bus, su dimensión en bits teniendo en cuenta que las señales de entrada A y D son de 4 bits y representan, respectivamente, valores con signo (en signo y magnitud) y sin signo (naturales).

Nota: Tenéis el ejercicio disponible en VerilCIRC. En el circuito, las señales T y s son también señales de salida para poder comprobar más fácilmente el funcionamiento. Tened en cuenta que VerilCIRC no soporta la verificación de subcircuitos diseñados por el usuario. Por lo tanto, tendréis que copiar en vuestra solución el circuito de la unidad de control diseñada en el apartado anterior.

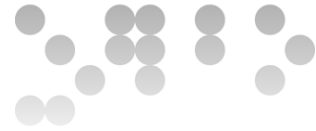
El camino de datos debe ocuparse de generar las señales necesarias para la unidad de control que, en este caso, son $(T>0)$ y $(T>D)$ y de actualizar y almacenar el valor de las variables que se usen y de generar los resultados de las operaciones que sean necesarias. En este caso, se usan las variables T y s , que es de 1 bit.

Las entradas A y D son de 4 bits y representan un valor binario en signo y magnitud el primero y natural el segundo. Por lo tanto, el rango de valores en decimal va de $-7_{(10)}$ a $+7_{(10)}$ en el caso de la entrada A , y de 0 a $15_{(10)}$ en el caso de la entrada D .

Con el valor de estas entradas se hace la operación $|A|+D$, para la que necesitaremos un sumador. El valor máximo que podemos tener como resultado de este cálculo es 21 (cuando D es 15 y $|A|$ es 7). El resultado se guarda en la variable T , que tendrá que ser de 5 bits para almacenarlo. Como A está en signo y magnitud, $|A|$ se consigue quitando el bit de signo y añadiendo ceros a la izquierda, si es necesario. Se usarán comparadores para obtener $(T>0)$ y $(T>D)$ que tendrán que ser de 5 bits.

Las entradas de selección (selectores) que provienen de la unidad de control eligen los valores siguientes para las variables T y s , y también para las señales L y R de la máquina de estados, tal como se muestra en la tabla siguiente, derivada de la de salidas que se da en el enunciado:

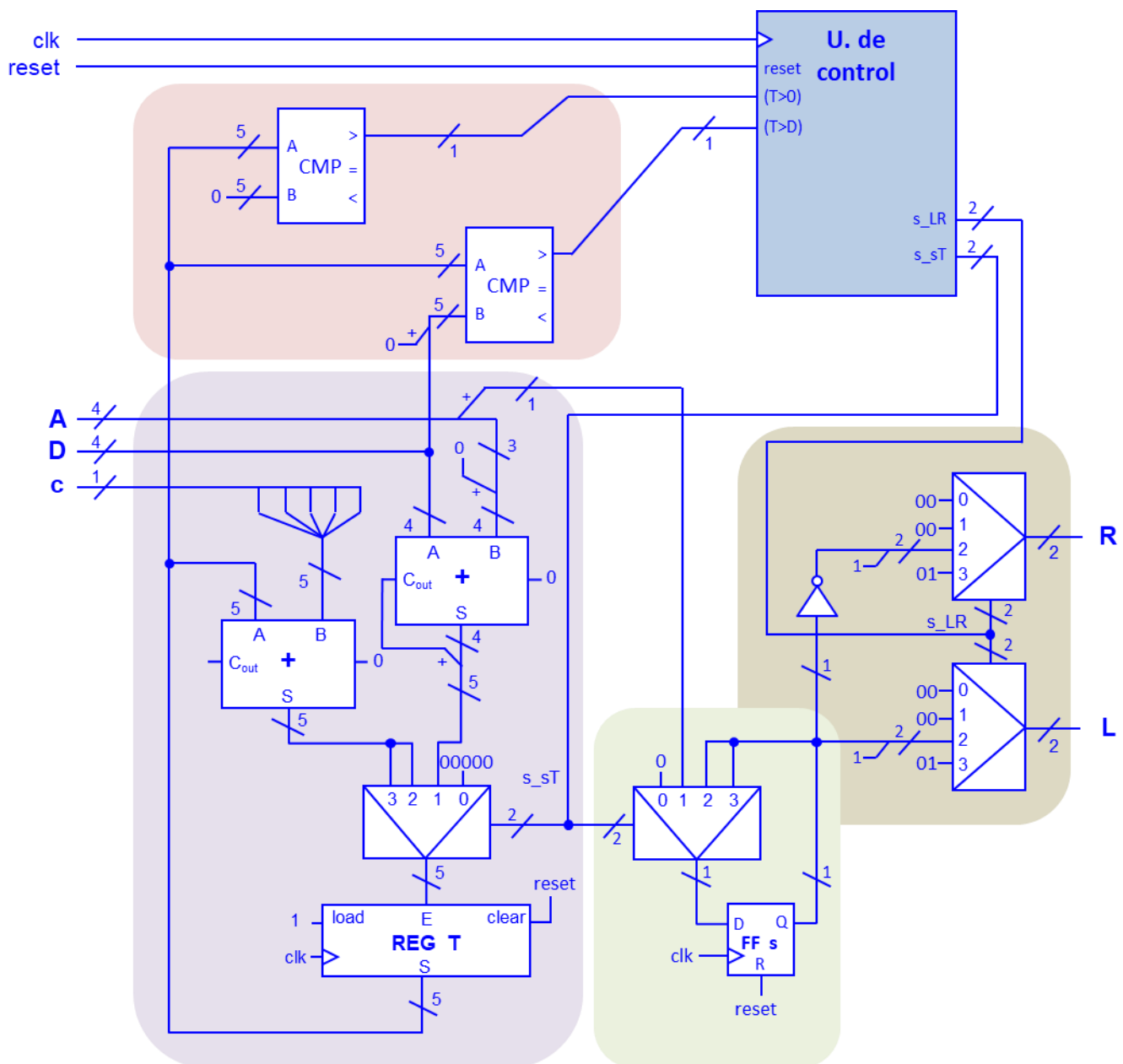
Señal o Variable y Selector	Estado			
	INIT	WAIT	TURN	FWD
$L =$	00	00	$1 - 2s$	01
$R =$	00	00	$2s - 1$	01
$s_{LR} =$	00	00	10	11
$s^+ =$	0	$\text{sign}(A)$	s	s
$T^+ =$	0	$ A + D$	$T - c$	$T - c$
$s_{sT} =$	00	01	10	10
Estado =	00	01	10	11

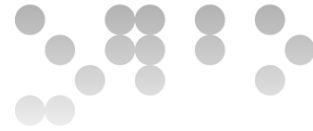


Las señales L y R son salidas primarias de dos bits. Según el valor del selector se les asigna 00, 01 o el resultado de la operación $1-2s$ en el caso de L y el de la operación $2s-1$ en el caso de R . La variable s es de 1 bit y, como se dice en el enunciado, en caso de que s sea 0, la primera operación da 1, y la segunda -1 . Esto en Ca2 y 2 bits es 01 y 11, respectivamente. Para la segunda operación, cuando s es 1 tenemos -1 y 1 , respectivamente, que en Ca2 y 2 bits es 11 y 01.

Si nos fijamos, el segundo bit es 1 en todos los casos. Sólo cambia el primero y lo hace de la forma siguiente: para $1-2s$ tiene el mismo valor que s y para $2s-1$ el valor contrario. Por tanto, se reduce a una selección del valor s o s' según el caso.

Con esto, el circuito completo para la EFSM de la fig. 2 queda de la forma siguiente:





SEGUNDA PARTE [35%]

Hay que construir el modelo del controlador de una caldera doméstica que proporciona agua corriente caliente y también agua caliente para el circuito de calefacción.

El elemento central de la caldera es un calentador al que llega agua fría de retorno de la calefacción o del intercambiador de calor secundario y la calienta para producir agua caliente para la calefacción o para calentar el agua fría de entrada y producir agua caliente sanitaria (ACS). Hay unas válvulas de paso que determinan qué operación se hace y que están gobernadas por la señal c . Si $c=0$ se produce agua caliente para la calefacción y si $c=1$, para el uso doméstico. Así pues, la caldera solo puede proporcionar un tipo de agua caliente o la otra.

El usuario puede establecer el modo de funcionamiento de la caldera a través de un módulo que proporciona la señal M . La caldera estará apagada si $M=0$, en modo de calefacción si $M=1$, preparada para producir agua caliente si $M=2$ y en modo de calefacción y de ACS si $M=3$. En este modo, la prioridad es para la ACS.

El calentador principal se activa cuando $b=1$ y la temperatura de referencia correspondiente, W (para la ACS) o H (para la calefacción) es superior a la medida, V o G , respectivamente.

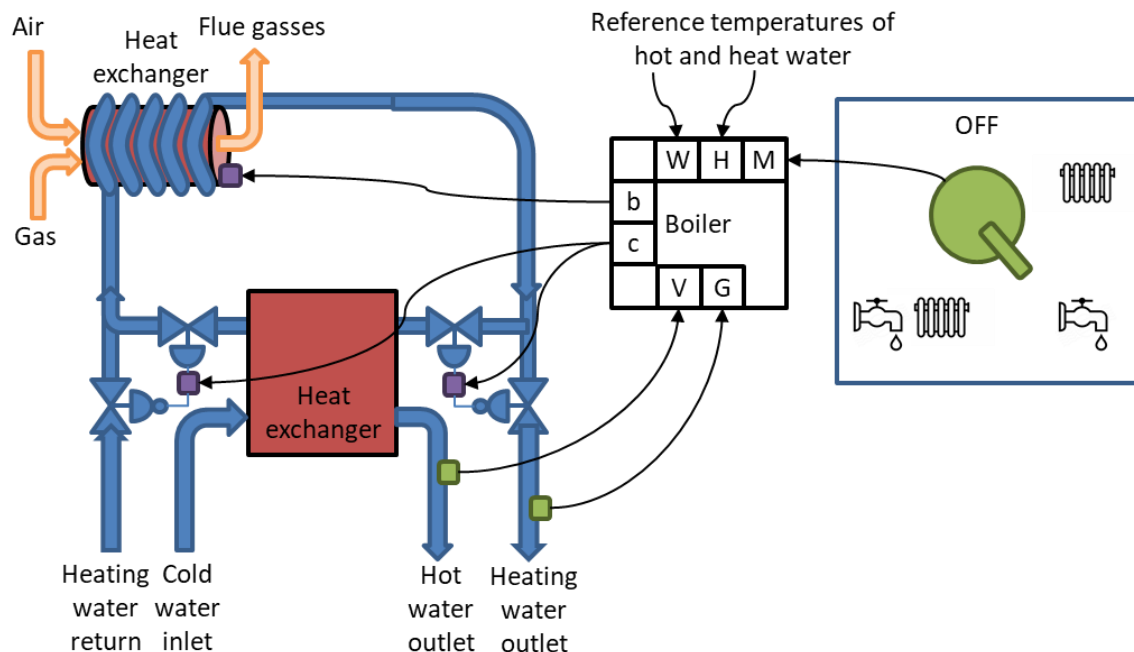
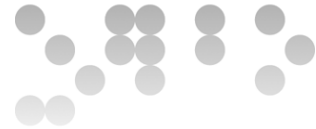


Fig. 3. Esquema simplificado de una caldera doméstica.

Para evitar que el calentador principal se encienda y se apague con demasiada frecuencia, una vez encendido, no se apagará hasta que la temperatura medida no supere la de referencia correspondiente en 5 grados. Por ejemplo, en modo $M=3$, si $H=50$ y $W=40$, con las temperaturas medidas siendo $G=49$ y $V=41$, pasaría a calentar el agua para la calefacción hasta que $G=55$ grados. Si, en este periodo, la temperatura de la ACS cae por debajo de 40, se cambiaría c a 1 y se esperaría a que la

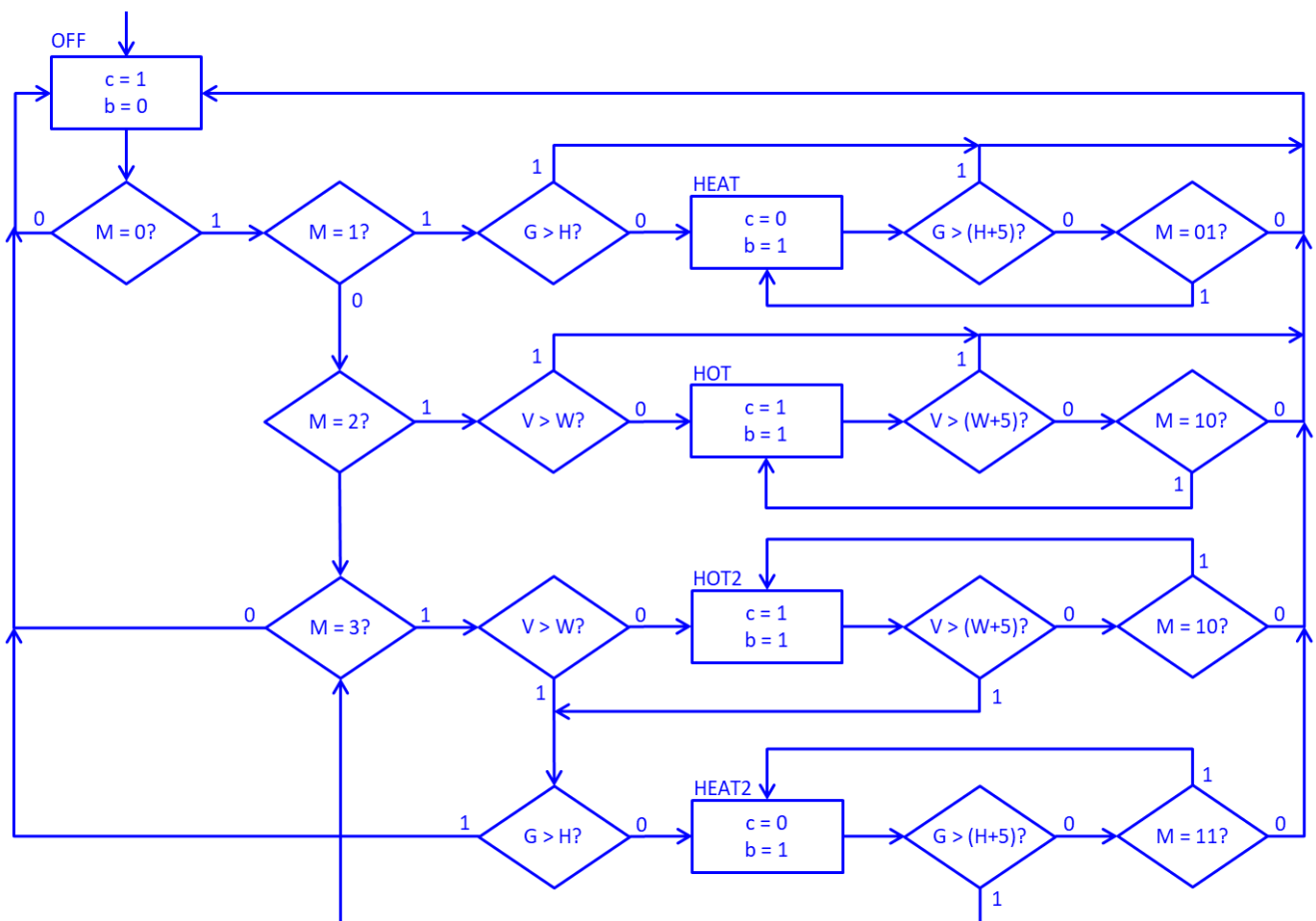


temperatura medida de V pasara a 45 grados. En ese momento, se volvería a conmutar el circuito ($c=0$) y se continuaría hasta que $G=55$ grados. Es decir, el cambio de modo no tiene que afectar las temperaturas a las que se tiene que llegar una vez se enciende el calentador.

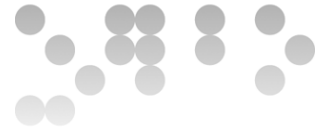
Diseñad la ASM del controlador de la caldera que se ha descrito.

El controlador tiene tres modos de funcionamiento, además del modo apagado. El modo de funcionamiento viene dado por el valor de M . Por lo tanto, podemos evaluar el valor de M y tratar cada modo de funcionamiento por separado.

Esto da lugar a un ASM como el de figura siguiente, donde la única particularidad es la prioridad que se tiene que establecer por el agua sanitaria cuando M tiene el valor 3:



En este caso no se hace necesario el uso de variables, porque en cada estado podemos establecer los valores correctos por las señales c y b . Podemos ver que en esta solución hay subconjuntos de estados y condiciones muy similares. Podemos usar una variable (K) para guardar la histéresis, según el caso, y simplificar la ASM.



Esta variable K se tiene que inicializar a 0. Toma el valor de la histéresis, el valor 5, cuando se pone en marcha el calentador ($b=1$) y lo mantiene hasta que se apaga ($b=0$). De esta forma, podemos reducir la ASM a la que aparece en la figura siguiente:

